

INTERNATIONAL POSTCODE SYSTEM (IPCS)

著者

バルトシュ・チェシリツキ (Bartosz Cieřlicki)

ロッチデール 2018 年

システムの前提条件

本システムの中心的なコンセプトは、地球全体を対象とする統一的な郵便番号システムを作ることです。最も重要な目的は次の通りです：

各郵便番号の地理座標を、その番号自体の記録の中にエンコードすること。

このように構築されたシステムは、各郵便番号の所在地を含む巨大なグローバルデータベースの必要性をなくします。また、ジオコーディングのコストを削減または完全になくし、要求された住所の地理位置を検索するコンピュータシステムを大幅に高速化します。

前提条件：

- 郵便番号が示す面積は 1000 m² を超えてはならない。これにより、地図や衛星ナビゲーションを使って目的地に容易かつ正確に到達できる。
- システムは人口、建物密度、地形、地表の種類にかかわらず、地球上のあらゆる場所をカバーしなければならない。
- 郵便番号はアルゴリズムに基づいて計算・生成されなければならない、任意に割り当てられてはならない。
- 各郵便番号の記録から、その面積の中心を示す地理座標を計算できる。

International Postcode System (IPCS) は次の条件を満たすべきである：

- 世界で最も一般的なラテンアルファベットを基礎とすること。
- 記憶しやすいように、比較的少ない文字数で構成されていること。
- 特定の国に固有の特殊文字を含まないこと。
- 記述・読取り時に発生しやすい最も一般的な間違い（文字 O と数字 0、文字 I と数字 1 の取り違いなど）を排除できること。

セクターの構成

International Postcode System (IPCS) を計算するには、十進法の度で表された緯度・経度を使用します。経度は $(-180, 180>$ の範囲、緯度は $(-90, 90>$ の範囲です。国際郵便番号を計算する最初の重要なステップは、緯度と経度を 0.001 の 1/4 単位で丸めることです。

例：

数 $t \in <0.000, 0.001>$ 、数 x を丸め値とします。次の場合：

$$x \in <0.000, 0.000125) \rightarrow x = 0.000$$

$$x \in <0.000125, 0.000375) \rightarrow x = 0.00025$$

$$x \in <0.000375, 0.000625) \rightarrow x = 0.0005$$

$$x \in <0.000625, 0.000875) \rightarrow x = 0.00075$$

$$x \in <0.000875, 0.001> \rightarrow x = 0.001$$

この丸め処理により、地球全体がゾーンに分割されます。得られた座標は、地球上に作られた各セクターを識別すると同時に、その中心点でもあります。これにより、儼 2 で述べた前提を満たします。

グリッドの抵粘は次のようになります：

座標に基づくセクターの構造上、緯度・経度の丸めによって形成される各セクターの面積は、地極に近づくにつれて小さくなります。したがって最大の面積は赤道上にあり、約 728 m² となり、これは前提条件を満たします。

この方法で作られるセクターの数を計算できます。数値は千分の 1 の 1/4 単位で丸められるため、各千分の 1 は 4 つに分割されます。

経度の値の範囲は $(-180, 180>$ です。x を分割数とすると、次のとおりです：

$$x = 360 * 1000 * 4 \quad (1)$$

緯度の値の範囲は $(-90, 90>$ です。y を分割数とすると、次のとおりです：

$$y = 181 * 1000 * 4 \quad (2)$$

(180 ではなく 181 を使うのは、緯度の範囲 $(-90, 90>$ が両端とも閉じていることを反映しています。南極 (-90°) と北極 (90°) は別々の 2 つの地点です。一方、経度の範囲 $(-180, 180>$ は片方の端だけが閉じており、 -180° と 180° は同じ子午線を指すためです。)

(1)と(2)を用いて、地球全体のセクター数を計算します：

$$\text{Number of sectors} = x * y = (360 * 1000 * 4) * (181 * 1000 * 4) = 1\,042\,560\,000\,000 \quad (3)$$

あとは、形成された各セクターに対して国際郵便番号 (IPCS) を構築するだけです。

International Postcode System (IPCS) の構築

ラテン文字と数字を使うと、10の数字と26の文字があります。システムの前提を満たすため、アルファベットから文字OとIを除きます。こうして次の34文字が得られます：{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, J, K, L, M, N, P, Q, R, S, T, U, V, W, X, Y, Z}。この34文字を基に、34進数体系を作ります。左側は34進数の数値、右側は十進法での対応値です。

0 = 0	8 = 8	G = 16	Q = 24	Y = 32
1 = 1	9 = 9	H = 17	R = 25	Z = 33
2 = 2	A = 10	J = 18	S = 26	
3 = 3	B = 11	K = 19	T = 27	
4 = 4	C = 12	L = 20	U = 28	
5 = 5	D = 13	M = 21	V = 29	
6 = 6	E = 14	N = 22	W = 30	
7 = 7	F = 15	P = 23	X = 31	

34進数の各数値は、十進法で次のように表されます：

$$\dots dcba_{34} = f(a) \times 34^0 + f(b) \times 34^1 + f(c) \times 34^2 + f(d) \times 34^3 + \dots$$

(fは34進数の各桁を十進法に対応させる関数です。(4)を参照。a, b, c, dは34進数の桁です。)

例：

$$\begin{aligned} AGZ15Z_{34} &= f(Z) \times 34^0 + f(5) \times 34^1 + f(1) \times 34^2 + f(Z) \times 34^3 + f(G) \times 34^4 + f(A) \times 34^5 \\ &= 33 \times 34^0 + 5 \times 34^1 + 1 \times 34^2 + 33 \times 34^3 + 16 \times 34^4 + 10 \times 34^5 \\ &= 33 \times 1 + 5 \times 34 + 1 \times 1156 + 33 \times 39304 + 16 \times 1336336 + 10 \times 45435424 \\ &= 33 + 170 + 1156 + 1297032 + 21381376 + 454354240 = 477034007 \end{aligned}$$

ご覧の通り、34進数表記ははるかに短くなります。地球全体の郵便番号を生成するには8文字で十分です。現行の英国・アイルランドの郵便番号の最大桁数は7、日本は7桁、オランダは6文字です。他国の郵便番号の長さ、International Postcode Systemがグローバルシステムであることを考慮し、本システムでは8文字が適切であり、前提条件を満たすと仮定します。

34進数で8桁で保存できる最大の数は（詳細な計算は省略）：

$$ZZZZZZZZ_{34} = 1\,785\,793\,904\,895$$

これにより、地球は最大1 785 793 904 896個の郵便番号を持ちうるようになります（00000000もコードとして使えるため、最大値より1大きい値）。先の計算(3)から、構造上のセクター数は1 042 560 000 000であることがわかっています。この数は1 785 793 904 895より小さいため、34進数で8桁あれば各セクターを一意に識別できます。セクターの面積は構造上均一ではなく、最大のものは赤道付近に位置し $\approx 728 \text{ m}^2$ です。地球の面積は約 $510\,100\,000 \text{ km}^2 (= 510\,100\,000\,000\,000 \text{ m}^2)$ です。セクターの平均面積

は次の計算で求められます：

$$510\,100\,000\,000\,000\,000\,\text{m}^2 / 1\,042\,560\,000\,000 \approx 489.2764\,\text{m}^2$$

したがって $ZZZZZZZZ_{34}$ は前提条件を満たし、各セクターの面積は $1000\,\text{m}^2$ 未満です。
残るは、セクター中心を International Postcode System (IPCS) に変換することだけです。
ここでは、負の数の整数部分を次のように表します：

- ・ 経度の場合、負の整数部分 x は $360 + x$ に変換されます。
- ・ 緯度の場合、負の整数 y は $180 + y$ に変換されます。

小数部分は変化しません。まとめると：

$$\text{Longitude } -XXX.xxxxx \rightarrow (360 + (-XXX)).xxxxx$$

$$\text{Latitude } -YYY.yyyyy \rightarrow (180 + (-YYY)).yyyyy$$

変換の例：

- ・ 経度 -75.43725 は 285.43725 となります ($360 + (-75) = 285$)。
- ・ 経度 -0.411250 は 360.411250 となります ($360 + 0 = 360$)。
- ・ 緯度 -15.44375 は 165.44375 となります ($180 + (-15) = 165$)。

アルゴリズムを作成する際は、この記録では緯度 90 度の南極と緯度 -90 度の北極が同一として扱われることに注意が必要です。

負の数の変換と丸め処理を考慮すると、十進法で記録される座標の一般形は次のとおりです：

Longitude: $XXX.xxxqq$, where $XXX \in \{x \in \mathbb{N}: 000 \leq x \leq 360\}$, $xxx \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}$, $qq \in \{00, 25, 50, 75\}$

Latitude: $YYY.yyygg$, where $YYY \in \{x \in \mathbb{N}: 000 \leq x \leq 180\}$, $yyy \in \{x \in \mathbb{N}: 000 \leq x \leq 999\}$, $gg \in \{00, 25, 50, 75\}$

次のステップでは、 $XXX.xxxqq$ と $YYY.yyygg$ の各要素を二進数に変換します。

$$XXX_{10} = XXXXXXXXX_2 \text{ — max. } 360, \text{ requires } 9 \text{ bits.}$$

$$xxx_{10} = xxxxxxxxx_2 \text{ — max. } 999, \text{ requires } 10 \text{ bits.}$$

$$qq_{10} = qq_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

$$YYY_{10} = YYYYYYYY_2 \text{ — max. } 180, \text{ requires } 8 \text{ bits.}$$

$$yyy_{10} = yyyyyyyyyy_2 \text{ — max. } 999, \text{ requires } 10 \text{ bits.}$$

$$gg_{10} = gg_2 \text{ — only 4 possible values: } 00_{10} \Rightarrow 00_2, 25_{10} \Rightarrow 01_2, 50_{10} \Rightarrow 10_2, 75_{10} \Rightarrow 11_2$$

次に、二進数を次の順序で並べます：

$$XXXXXXXXXX\,YYYYYYYY\,xxxxxxxxxx\,yyyyyyyyyy\,qq\,gg_2$$

最低 41 ビットの数を用意してゼロにし、上記のビットを順序を保ったまま右から左へ入力します。得られる数は次のとおりです：

$$XXXXXXXXXXYYYYYYYYxxxxxxxxxyyyyyyyyqqgg_2$$

最後のステップで、二進数を 34 進数に変換します。得られた 34 進数が、そのまま国際郵便番号 (IPCS) となります。セクター中心の座標は、この国際郵便番号 (IPCS) の記録内にエンコードされています。

ビットを書く順序は非常に重要です。この規則に従って丸め・表示した場合、最大

の数値を持つ座標は 360.99975, 180.99975 です。これを二進数に変換すると次の数が得られます：

$$10110100010110100111110011111110011111112 = 1\,549\,224\,509\,055_{10}$$

得られた数は、34 進数の最大値である $ZZZZZZZ_{34} = 1\,785\,793\,904\,895_{10}$ より小さい値です。ビットの順序を変えるとこの最大値を超えてしまう可能性があり、すべてのセクターに郵便番号を作成できなくなってしまう。

セクター中心（国際郵便番号）の座標を得るには、逆の順序で操作を行います。

例:

座標 -1.347120, 53.983489 に対する国際郵便番号 (IPCS) は次のように構築します:

丸め処理して座標を変換し、359.34700, 53.98350 を得ます。

座標の各要素 (すなわち数値 359, 347, 00, 53, 983, 50) を上記の方法で二進数に変換し、次を得ます: 101100111_2 , 0101011011_2 , 00_2 , 01101010_2 , 1111010111_2 , 10_2

数値を正しい順序に並べます: 101100111_2 , 0011010_{21} , 0101011011_2 , 1111010111_2 , 00_2 , 10_2

数 $1011001110011010101010101101111110101110010_2$ を作成します。

二進数を 34 進数に変換し、 $VCPM6TKY_{34}$ を得ます。

得られた数が国際郵便番号 (IPCS) です: $VCPM6TKY$

国際郵便番号 1GL8 YT7F の座標は次のように計算します:

34 進数 $1GL8YT7F_{34}$ を二進数に変換し、 $000010010001100101100000011010011011010012$ を得ます。

数値を抽出します: 00001001_2 , 0011001_{20} , 1100000011_2 , 0100110110_2 , 10_2 , 01_2

順序を 00001001_2 , 1100000011_2 , 10_2 , 0011001_{20} , 0100110110_2 , 01_2 に変えます。

二進数を十進数に変換します: 18, 771, 50, 50, 310, 25

得られた数値から座標 18.77150, 50.31025 が得られます。

国際郵便番号から座標を生成・計算する関数

以下は C# で書かれた 2 つの関数です。最初の関数は国際郵便番号を構築し、次の関数は国際郵便番号から座標を計算します。

```
public string GeneratePostcode(double Lon, double Lat)
{
    if ((Lon > 180) || (Lon < -180) || (Lat > 90) || (Lat < -90)) return "";
    if (Lat > 89.99975) Lat = 89.99975;
    if (Lat < -89.99975) Lat = -89.99975;

    string[] Coder = { "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E", "F", "G", "H", "J",
        "K", "L", "M", "N", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z" };

    if (Lat < 0)
    {
        Lat = ((long)Lat + 180) + Math.Abs(Lat - (long)Lat);
    }
    if (Lon < 0)
    {
        Lon = ((long)Lon + 360) + Math.Abs(Lon - (long)Lon);
    }

    string PartPostcodeLat = Lat.ToString("000.000000");
    string PartPostcodeLon = Lon.ToString("000.000000");

    try
    {
        string[] StrsLat = PartPostcodeLat.Split('.');
        string[] StrsLon = PartPostcodeLon.Split('.');

        ulong OptionsLat = 5;
        long Precision = Convert.ToInt32(StrsLat[1].Substring(3));
        if (Precision < 125)
        {
            OptionsLat = 0;
        }
        else if (Precision < 375)
        {
            OptionsLat = 1;
        }
        else if (Precision < 625)
        {
            OptionsLat = 2;
        }
        else if (Precision < 875)
        {
            OptionsLat = 3;
        }

        ulong TempHiLat = Convert.ToUInt64(StrsLat[0]);
        TempHiLat = TempHiLat << 24;
        ulong TempLoLat = 0;
        if (StrsLat.Length > 1)
        {
            TempLoLat = Convert.ToUInt64(StrsLat[1].Substring(0, 3));
            if (OptionsLat == 5)
            {
                TempLoLat++;
            }
            else
            {
                TempHiLat = TempHiLat | OptionsLat;
            }
        }
    }
}
```

```

        TempLoLat = TempLoLat << 4;
    }

    ulong OptionsLon = 5;
    Precision = Convert.ToInt32(StrsLon[1].Substring(3));
    if (Precision < 125)
    {
        OptionsLon = 0;
    }
    else if (Precision < 375)
    {
        OptionsLon = 1;
    }
    else if (Precision < 625)
    {
        OptionsLon = 2;
    }
    else if (Precision < 875)
    {
        OptionsLon = 3;
    }

    ulong TempHiLon = Convert.ToUInt64(StrsLon[0]);
    TempHiLon = TempHiLon << 32;
    ulong TempLoLo = 0;
    if (StrsLon.Length > 1)
    {
        TempLoLo = Convert.ToUInt64(StrsLon[1].Substring(0, 3));
        if (OptionsLon == 5)
        {
            TempLoLo++;
        }
        else
        {
            TempHiLon = TempHiLon | (OptionsLon * 4);
        }
    }

    TempLoLo = TempLoLo << 14;
}

ulong UlongResHi = TempHiLat | TempHiLon | TempLoLat | TempLoLo;

ulong Pos = 0;
string Res = "";
for (int i = 7; i >= 0; i--)
{
    Pos = UlongResHi / (ulong)Math.Pow(34, i);
    UlongResHi -= (Pos * (ulong)Math.Pow(34, i));
    Res += Coder[Pos];
}

return Res;
}
catch
{
    return "";
}
}

public bool ReversePostcode(string Postcode, out double Lat, out double Lon)
{
    if (Postcode.Length == 8)
    {
        Postcode = Postcode.ToUpper();
        string Coder = "0123456789ABCDEFGHIJKLMNPQRSTUVWXYZ";
    }
}

```

```

ulong Hi = 0;

for (int i = 0; i < 8; i++)
{
    Hi += (ulong)Coder.IndexOf(Postcode[i]) * (ulong)Math.Pow(34, (7 - i));
}

ulong OptionLat = (Hi & 3);
double HalfLat = 0;
switch (OptionLat)
{
    case 1:
        HalfLat = 0.00025;
        break;
    case 2:
        HalfLat = 0.0005;
        break;
    case 3:
        HalfLat = 0.00075;
        break;
}

ulong OptionLon = (Hi & 12) >> 2;
double HalfLon = 0;
switch (OptionLon)
{
    case 1:
        HalfLon = 0.00025;
        break;
    case 2:
        HalfLon = 0.0005;
        break;
    case 3:
        HalfLon = 0.00075;
        break;
}

long LoLat = ((long)Hi >> 4) & 1023;
long HiLat = ((long)Hi >> 24) & 255;
if (HiLat > 180)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLat > 90)
{
    HiLat = HiLat - 180;
    Lat = HiLat - ((LoLat / 1000.0) + HalfLat);
}
else
{
    Lat = HiLat + ((LoLat / 1000.0) + HalfLat);
}

long LoLon = ((long)Hi >> 14) & 1023;
long HiLon = ((long)Hi >> 32);
if (HiLon > 360)
{
    Lat = -180;
    Lon = -180;
    return false;
}
if (HiLon > 180)
{
    HiLon = HiLon - 360;
}

```

```
        Lon = HiLon - ((LoLon / 1000.0) + HalfLon);
    }
    else
    {
        Lon = HiLon + ((LoLon / 1000.0) + HalfLon);
    }

    return true;
}
else
{
    Lat = 0;
    Lon = 0;
    return false;
}
}
```